

Національний університет «Одеська Політехніка»

Інститут комп'ютерних систем

Кафедра інформаційних систем

Лабораторна робота №3

з дисципліни «Програмування мобільних пристроїв на базі ОС Android»

Тема: «Навігація в Jetpack Compose»

Виконав:

студент групи AI-234

Середа Я. В.

Перевірили:

Годовиченко М. А.

Соловйова Д. В.

Одеса 2026

МЕТА РОБОТИ

Ознайомитися з принципами побудови багатоекранних мобільних застосунків з використанням декларативного фреймворку Jetpack Compose та компонента Navigation Compose. Здобути практичні навички налаштування маршрутизації, переходів між екранами та передачі аргументів. Закріпити знання щодо управління спільним станом UI через ViewModel.

ЗАВДАННЯ

- Розробити багатоекранний Android-застосунок «Мої завдання» (варіант 4) з трьома екранами: список завдань, форма додавання/редагування та деталі завдання.
- Налаштувати навігаційний граф (NavHost) для управління переходами між екранами за допомогою `NavController`.
- Реалізувати передачу аргументів між маршрутами (передача ID завдання на екран деталей у форматі `"details/{taskId}"`).
- Забезпечити управління даними через ViewModel (додавання, редагування, зміна статусу виконання та видалення завдань).

1 ХІД РОБОТИ

1.1 Проектування моделі даних та стану

Створено `data class TaskItem`, який містить поля `id`, `title`, `description`, `isCompleted`, `dueDate` та перелічення `TaskPriority`. Для збереження стану застосунку розроблено класи `TasksListUiState`, `AddEditTaskUiState` та `TaskDetailsUiState`.

1.2 Розробка ViewModel

Створено клас `TasksViewModel`, який відповідає за збереження масиву завдань у пам'яті за допомогою `MutableStateFlow`. У ViewModel реалізовано методи для підготовки стану під час переходів: `prepareNewTask()`, `prepareEditTask()` та `prepareDetails()`. Також додано метод `saveTask()`, у якому реалізовано логіку автоматичного призначення терміну виконання (`dueDate`) на основі обраного пріоритету (наприклад, "Високий" -> "До завтра").

1.3 Налаштування навігації

Створено об'єкт `AppRoutes` для зберігання констант маршрутів. Налаштовано компонент `NavHost`, в якому визначено три основні маршрути (`composable`). Для екранів редагування та деталей налаштовано прийом аргументу `taskId` через `navArgument` з типом `NavType.IntType`. За допомогою `LaunchedEffect` реалізовано виклик методів ViewModel для завантаження конкретного завдання при переході на екран деталей.

1.4 Побудова інтерфейсу (UI)

Реалізовано три екрани:

TasksListScreen: відображає список завдань через `LazyColumn`. Кожна картка містить кольоровий індикатор пріоритету та бейдж. Додано візуальне відображення виконаних завдань (напівпрозорість та закреслений текст).

AddEditTaskScreen: форма з полями `OutlinedTextField` та випадаючим списком `ExposedDropdownMenuBox` для вибору пріоритету.

TaskDetailsScreen: відображає розширену інформацію про завдання, містить чекбокс для відмітки виконання та кнопки для редагування або видалення.

2 ПЕРЕВІРКА ФУНКЦІОНАЛЬНОСТІ ДОДАТКУ

2.1 Перевірка головного екрана зі списком завдань

При запуску додатка успішно ініціалізується екран `TasksListScreen` із попередньо заповненим списком. Елементи відображаються коректно, з відповідними кольоровими індикаторами пріоритетів (рис. 2.1).

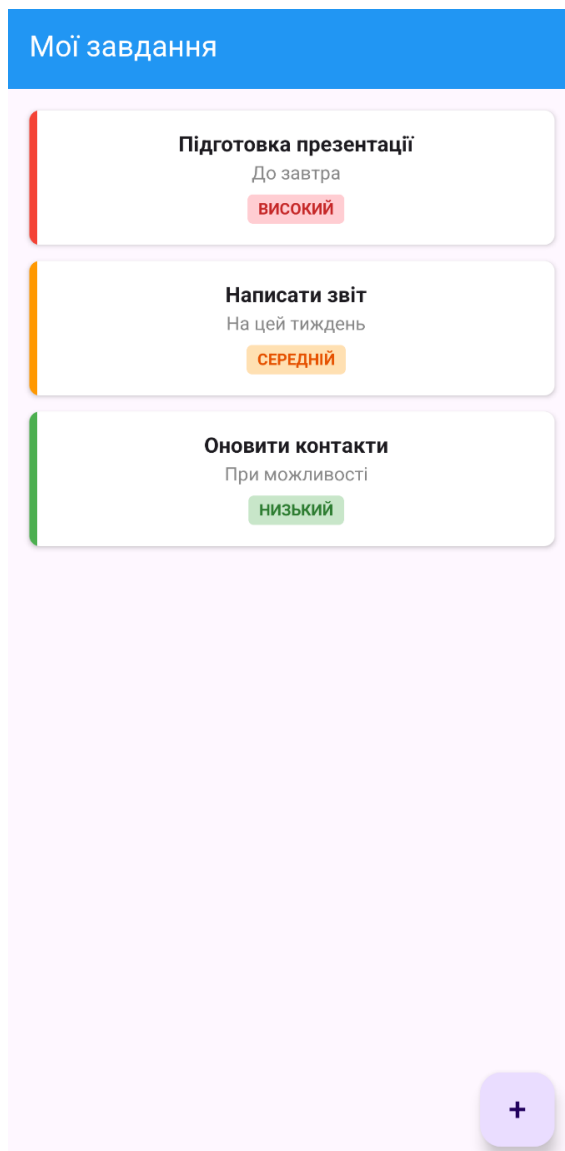
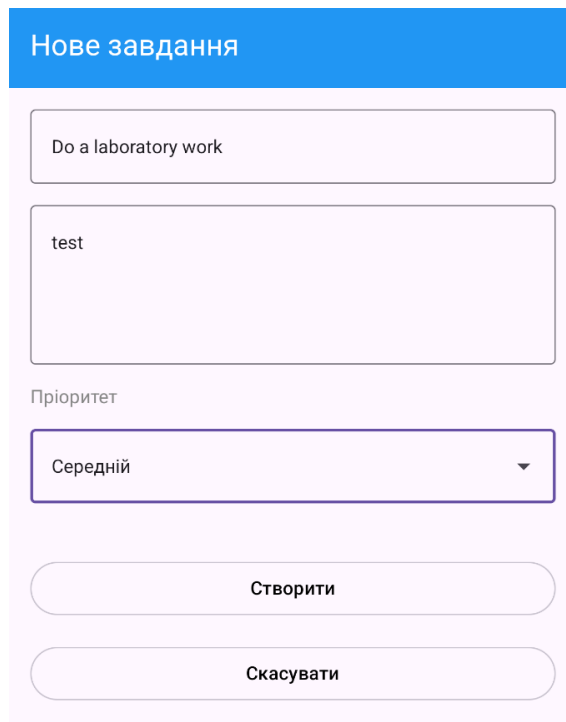


Рисунок 2.1 – Початковий стан головного екрана зі списком завдань

2.2 Додавання нового завдання

При натисканні на кнопку "+" відбувається перехід на екран **AddTaskScreen**. Форма дозволяє ввести назву, опис та обрати пріоритет із випадаючого списку (рис. 2.2).



Нове завдання

Do a laboratory work

test

Пріоритет

Середній

Створити

Скасувати

Рисунок 2.2 – Екран створення нового завдання з відкритим меню вибору пріоритету

Після натискання кнопки «Створити», застосунок використовує **popBackStack()**, повертаючи користувача на головний екран. Нове завдання "Do a laboratory work" з'являється у списку. Термін «На цей тиждень» розраховано автоматично на основі середнього пріоритету (рис. 2.3).

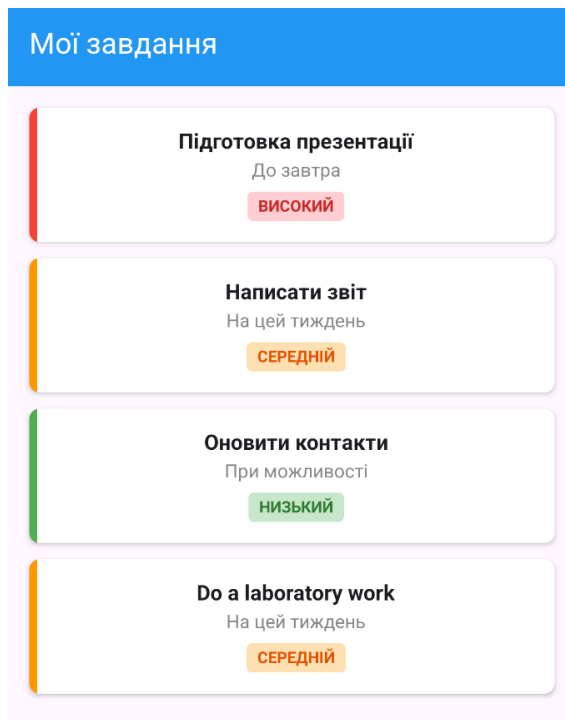


Рисунок 2.3 – Відображення нового завдання у загальному списку

2.3 Перехід до деталей завдання та відмітка про виконання

Натискання на створене завдання успішно передає його ID через параметри маршруту до екрана `TaskDetailsScreen`. На екрані відображаються коректні дані (рис. 2.4). Натискання на чекбокс «Позначити як виконане» оновлює статус у ViewModel (рис. 2.5).

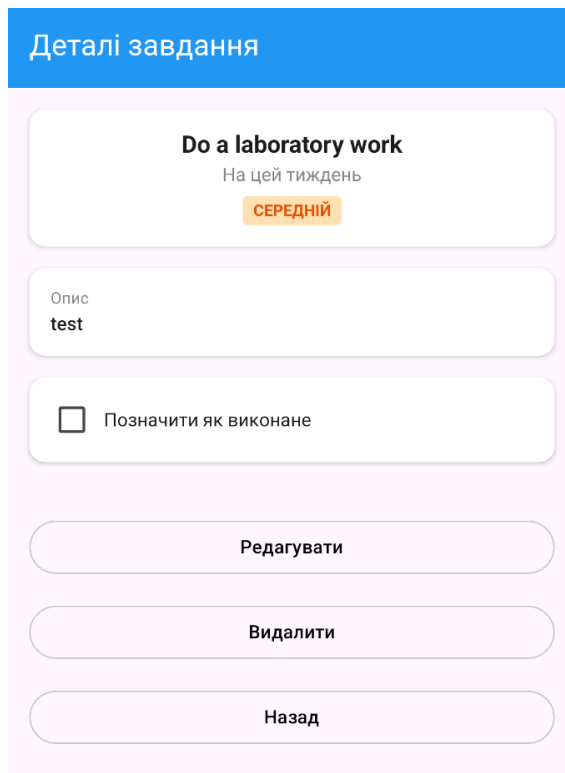


Рисунок 2.4 – Екран деталей завдання з функцією зміни статусу виконання

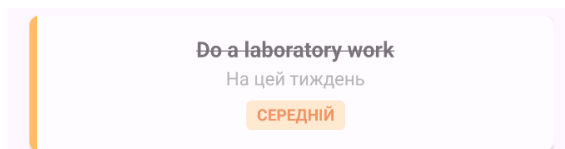


Рисунок 2.5 – Натиснутий чекбокс «Позначити як виконане»

2.4 Редагування завдання

При натисканні на кнопку «Редагувати» відбувається перехід до екрана форми з передачею поточних даних. Зміна назви завдання на "Do a course work" та пріоритету на «Високий» коректно зберігається та автоматично оновлює термін на «До завтра» (рис. 2.6).

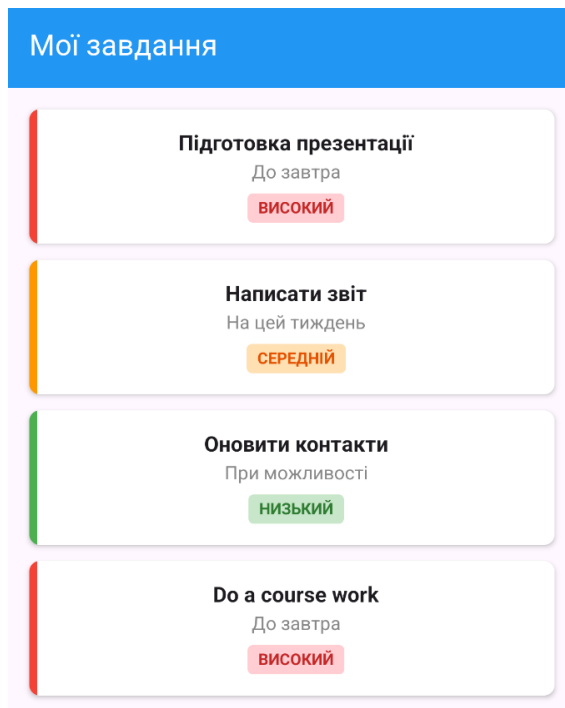


Рисунок 2.6 – Зміна назви та пріоритету

2.5 Видалення завдання

На екрані деталей натискання кнопки «Видалити» ініціює видалення елемента з `StateFlow` у `ViewModel` та повернення на головний екран. Завдання повністю зникає зі списку (рис. 2.7).

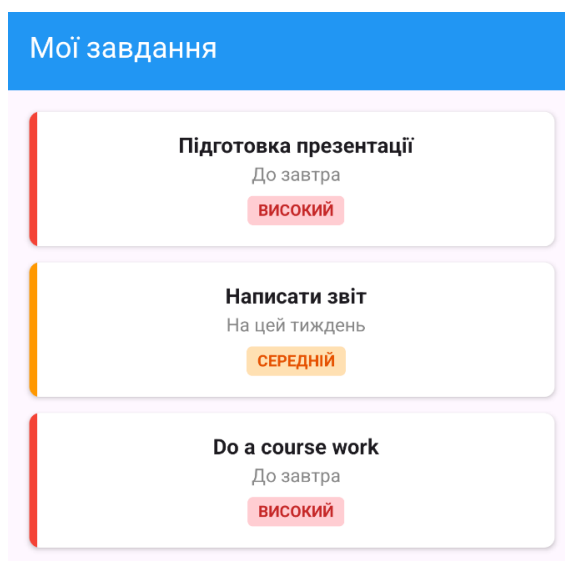


Рисунок 2.7 – Головний екран після успішного видалення завдання

ВИСНОВОК

Під час виконання лабораторної роботи було успішно розроблено багатоекранний Android-застосунок з повноцінною системою навігації на базі Jetpack Compose. Практично закріплено навички роботи з `NavHost`, `NavController` та передачі аргументів між маршрутами.

Архітектура застосунку з використанням єдиної ViewModel дозволила ефективно синхронізувати стан даних між трьома різними екранами, забезпечивши миттєве оновлення інтерфейсу при створенні, редагуванні, зміні статусу та видаленні завдань.